

Single Source Shortest Path

Day 4 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“minimum distance from node s ”*
- ▶ *“if there is no path from s to that node”*

Programming interpretation:

- ▶ Each test case is a directed graph with nonnegative edge weights.
- ▶ Run Dijkstra once from the source, then answer all destination queries from dist.
- ▶ The all-zero line ends the input.

Textbook connection: CP4 4.4.3 Dijkstra's Algorithm

Core technique: Standard Dijkstra with query answering

- ▶ Each test case is a directed weighted graph.
- ▶ All edge weights are nonnegative.
- ▶ After one source is fixed, multiple queries ask for distances from that source.

- ▶ Nonnegative weighted shortest paths point to Dijkstra.
- ▶ Multiple queries from the same source should be answered after one run.
- ▶ An adjacency list stores only the given directed edges.

- ▶ Read V , E , q , and source s until 0 0 0 0.
- ▶ Build $AL[u]$ as a list of (v, w) directed edges.
- ▶ Initialize $dist[s] = 0$ and all others to INF .
- ▶ Use a priority queue to process vertices by smallest known distance.
- ▶ Relax outgoing edges and answer each query from the final $dist$ array.

- ▶ A stale priority queue entry has $d > \text{dist}[u]$ and can be skipped.
- ▶ Unreachable query vertices remain at INF.
- ▶ The output requires a blank line after each test case.

- ▶ Dijkstra's invariant finalizes each popped non-stale vertex at its true shortest distance.
- ▶ Every directed edge is relaxed when its source is finalized or improved.
- ▶ The distance array therefore answers all post-run queries correctly.

Complexity: $O((V + E) \log V)$ time per test case and $O(V + E)$ memory.

- ▶ Do not add reverse edges; the graph is directed.
- ▶ Use a large integer INF instead of float infinity for integer distances.

Source lines: shortestpath1.py:48-65

```
import sys
from heapq import heappush, heappop

def main():
    input = sys.stdin.buffer.readline

    INF = int(1e18)

    output = []

    while True:
        V, E, q, s = map(int, input().split())

        if V == 0 and E == 0 and q == 0 and s == 0:
            break
```

What this chunk does

- ▶ The loop continues until the sentinel 0 0 0 0.
- ▶ INF represents vertices that have not been reached.
- ▶ output buffers answers across test cases.

Source lines: `shortestpath1.py:67-78`

```
AL = [[] for u in range(V)]

for _ in range(E):
    u, v, w = map(int, input().split())

    AL[u].append((v, w))
```

What this chunk does

- ▶ `AL[u]` stores only edges leaving `u`.
- ▶ The edge direction matters; `v` to `u` is not added unless it appears separately.
- ▶ Each stored pair is destination and weight.

Source lines: `shortestpath1.py:79-93`

```
dist = [INF for u in range(V)]  
  
dist[s] = 0  
  
pq = []  
heappush(pq, (0, s))
```

What this chunk does

- ▶ All distances start at INF except the source distance of zero.
- ▶ The heap orders vertices by best known distance.
- ▶ The initial heap entry starts the search from s.

Source lines: `shortestpath1.py:95-122`

```
while len(pq) > 0:
    d, u = heappop(pq)

    if d > dist[u]:
        continue

    for v, w in AL[u]:

        new_distance = dist[u] + w

        if new_distance >= dist[v]:
            continue

        dist[v] = new_distance

        heappush(pq, (dist[v], v))
```

What this chunk does

- ▶ The smallest queued distance is popped first.
- ▶ Stale heap entries are ignored.
- ▶ Each edge relaxation improves $\text{dist}[v]$ and pushes the new candidate distance.

Source lines: `shortestpath1.py:124-143`

```
for _ in range(q):
    vertex = int(input())

    if dist[vertex] == INF:
        output.append("Impossible")
    else:
        output.append(str(dist[vertex]))

    output.append("")

sys.stdout.write("\n".join(output))

main()
```

What this chunk does

- ▶ After Dijkstra, each query is just one array lookup.
- ▶ Unreached vertices still have INF and print Impossible.
- ▶ A blank line is added after each test case as required.

Solution file: `practice_contests/day_4/shortestpath1.py`

Problem: <https://open.kattis.com/problems/shortestpath1>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.